

Modélisation et Conception Orientée-Objet

Frédéric Dadeau

Laboratoire d'Informatique de l'université de Franche-Comté
Bureau 421C
`frederic.dadeau@lifc.univ-fcomte.fr`

Licence 3 Informatique

Plan du cours

- 1 Présentation du module MCOO
 - M.C.O.O.
 - Méthodes de développement
 - Concepts objet
 - U.M.L.
- 2 Les diagrammes des aspects statiques
 - Diagrammes de cas d'utilisation
 - Diagrammes de classes et d'objets
 - Diagrammes de composants
 - Diagrammes de déploiement
- 3 Les diagrammes des aspects dynamiques
 - Diagrammes de séquence
 - Diagrammes de collaboration
 - Diagrammes d'états-transition
 - Diagrammes d'activité

M.C.O.O. qu'est-ce que c'est ?

Modélisation et

Conception

Orienté

Objet

M.C.O.O. qu'est-ce que c'est ?

Modélisation et
établir un modèle

Conception
élaborer, se représenter quelque chose

Orienté

Objet
vers l'objet

MCOO : modéliser et concevoir en utilisant les objets du monde réel.

Cours : 3 séances + 1 interrogation écrite

- lundi 14 septembre – de 13h30 à 15h
- mardi 15 septembre – de 13h30 à 15h
- lundi 21 septembre – de 11h à 12h30

Travaux dirigés : 10 séances (dont 1 interrogation écrite)

- TD1 : mardi de 15h à 16h30
- TD2 : mardi de 13h30 à 15h

Travaux pratiques : 1 séance de 1h30 la semaine du 30 novembre

- TP A : jeudi de 13h30 à 15h
- TP AB : lundi de 9h30 à 11h
- TP B : lundi de 16h30 à 18h

Travail personnel : étude de cas à travailler en autonomie (créneaux de TP réservés)

Historique des méthodes de développement

Liées aux langages de programmations.

Les méthodes fonctionnelles (1970 –)

Décomposition d'une fonction en plusieurs sous-fonctions.

- Architecture basée sur la décomposition fonctionnelle : peu d'évolutions possibles
- Peu de réutilisation des composants

Les méthodes classiques (1980 –)

Modélisation des données et des traitements.

- JSD : Jackson System Development
- MACH : Méthode d'Analyse et de Conception Hiérarchisée
- MERISE : Méthode d'Etude et de Réalisation Informatique pour les Systèmes d'Entreprise
- SADT : Structured Analysis and Design Technique
- SASD : Structured Analysis Structured Design

Historique des méthodes de développement

Liées aux langages de programmations.

Les méthodes objet

Objet = abstraction des entités du monde réel

- aide à visualiser les objets tel qu'ils sont ou tels qu'on le souhaite
- permet de spécifier la structure ou le comportement d'un objet
- fournit un guide pour la construction du système
- documente les décisions prises lors de la construction d'un système

Quelques méthodes orientées-objet

- OOA : Object Oriented Analysis, Schlaer/Mellor (1988,91,93)
- OOD : Object Oriented Design, Booch (1991,93)
- OMT : Object Modelling Technique, Rumbaugh (1991,95)
- OOSE : Object Oriented Software Engineering, Jacobson (1992)

Unification des méthodes

Unified Method

- auteurs : Booch, Rumbaugh et Jacobson
- rapprochement de leurs méthodes + cas d'utilisation

Historique de la création d'UML

- 1995 Début du travail sur la méthode unifiée
- 1996 Création d'un consortium de partenaires pour travailler sur la définition d'UML dans l'OMG
- 1997 UML 1.1 (normalisé par l'OMG)
- 1998 UML 1.2
- 1999 UML 1.3
- 2001 UML 1.4
- 2003 UML 1.5
- 2005 UML 2.0

Unification des méthodes

Unified Method

- auteurs : Booch, Rumbaugh et Jacobson
- rapprochement de leurs méthodes + cas d'utilisation

Object Management Group – OMG

- Organisation internationale créée en 1989
- Plus de 800 membres (informaticiens et utilisateurs)
- Vise à promouvoir la théorie et la pratique de la technologie objet dans le développement logiciel
- Egalement à la base des standards Meta-Object Facility (MOF), COmmon Request Broker Architecture (CORBA) et Interface Definition Language (IDL)

UML : Unified Modeling Language

- Notation graphique de modélisation à objets
- Permet de visualiser, spécifier, construire et documenter les différentes parties d'un système logiciel
- Langage graphique, basé sur des diagrammes (9 pour UML 1.5, 13 pour UML 2.0)
- **Pas une méthode !** mais utilisable dans tout le cycle de développement d'un logiciel

UML : Unified Modeling Language

- Notation graphique de modélisation à objets
- Permet de visualiser, spécifier, construire et documenter les différentes parties d'un système logiciel
- Langage graphique, basé sur des diagrammes (9 pour UML 1.5, 13 pour UML 2.0)
- **Pas une méthode !** mais utilisable dans tout le cycle de développement d'un logiciel

Buts initiaux des concepteurs d'UML

- représenter des systèmes entiers (pas uniquement logiciels) par des concepts objets ;
- lier explicitement des concepts et le code qui les implantent ;
- pouvoir modéliser des systèmes à différents niveaux de granularité (pour permettre d'appréhender des systèmes complexes) ;
- créer un langage de modélisation utilisable à la fois par les humains et les machines.

L'objet

Entité du monde réel (ou virtuel) centralisant les données et les traitements associés.

Objet = Identité + Etat + Comportement

L'objet

Entité du monde réel (ou virtuel) centralisant les données et les traitements associés.

Objet = Identité + Etat + Comportement

Identité d'un objet

Propriété qui permet de distinguer un objet par rapport aux autres sans ambiguïtés.
Distingue deux objets dont les valeurs des différents attributs sont identiques.

L'objet

Entité du monde réel (ou virtuel) centralisant les données et les traitements associés.

Objet = Identité + Etat + Comportement

Identité d'un objet

Propriété qui permet de distinguer un objet par rapport aux autres sans ambiguïtés.
Distingue deux objets dont les valeurs des différents attributs sont identiques.

Etat d'un objet

Valeur de tous ses attributs à un état donné.
Un attribut est une information qui qualifie l'objet qui la contient.

L'objet

Entité du monde réel (ou virtuel) centralisant les données et les traitements associés.

Objet = Identité + Etat + Comportement

Identité d'un objet

Propriété qui permet de distinguer un objet par rapport aux autres sans ambiguïtés.
Distingue deux objets dont les valeurs des différents attributs sont identiques.

Etat d'un objet

Valeur de tous ses attributs à un état donné.
Un attribut est une information qui qualifie l'objet qui la contient.

Comportement d'un objet

Définit l'ensemble des opérations que l'objet peut exécuter en réaction aux messages envoyés par les autres objets.
L'état et le comportement sont liés.

La notion de classe

Une classe est une abstraction qui représente un ensemble d'objets de même nature, c'est-à-dire possédant la même structure statique (attributs) et le même comportement (méthodes).

Exemple : classe Point2D avec les attributs x et y .

Relation entre objet et classe

- Tout objet appartient à une classe et connaît de façon implicite la classe à laquelle il appartient.
- Tout objet est une instance de sa classe.
- Un objet est une instance d'une et une seule classe.

Classe abstraite

Classe conçue sans instances.

Relations entre classes

Association

Relation entre plusieurs classes.

Elles représentent une abstraction des différents liens qui peuvent exister entre les différents objets.

Agrégation et composition

- Forme particulière d'association entre deux classes.
- Exprime le fait qu'une classe est composée d'une ou plusieurs autres classes.

Spécialisation et généralisation

Le mécanisme d'héritage permet de définir de nouvelles classes à partir de classes déjà existantes.

- Généralisation : factorisation dans une classe (appelée super-classe) de propriétés de plusieurs classes
- Spécialisation : inverse de la généralisation, consiste à créer à partir d'une classe plusieurs classes spécialisées.

Notions relatives aux composants des classes

Encapsulation

Possibilité de masquer certains détails de l'implantation. Peut-être réalisé en particuliers par des constituants "privés" des objets.

Polymorphisme

Poly = plusieurs, morphisme = forme

Caractéristique d'un élément qui peut se présenter sous plusieurs formes.

Capacité donnée à une même opération de s'effectuer différemment suivant le contexte de la classe où elle se trouve.

Les diagrammes d'UML

Un diagramme donne à l'utilisateur la possibilité de visualiser et de manipuler des éléments de modélisation.

UML 2.0 comporte 13 diagrammes permettant de modéliser différents aspects d'un système. On se focalisera dans ce cours sur les 9 diagrammes issus d'UML 1.5.

2 catégories de diagrammes UML

- Aspects statiques (structure logicielle et physique) – 5 diagrammes
- Aspects dynamiques (comportements) – 4 diagrammes

Les diagrammes d'UML

Un diagramme donne à l'utilisateur la possibilité de visualiser et de manipuler des éléments de modélisation.

UML 2.0 comporte 13 diagrammes permettant de modéliser différents aspects d'un système. On se focalisera dans ce cours sur les 9 diagrammes issus d'UML 1.5.

Diagrammes des aspects statiques

- 1 Diagramme de **cas d'utilisation** : représente les fonctions du système du point de vue de l'utilisateur.
- 2 Diagramme de **classes** : définit les classes et les relations entre les classes.
- 3 Diagramme d'**objets** : définit les instances des classes, ainsi que les liens entre ces instances.
- 4 Diagramme de **composants** : permet de décrire les composants (ou modules) d'un système (fichiers sources, fichiers objets, bibliothèques exécutables).
- 5 Diagramme de **déploiement** : décrit la disposition physique du matériel et la répartition des composants sur le matériel.

Les diagrammes d'UML

Un diagramme donne à l'utilisateur la possibilité de visualiser et de manipuler des éléments de modélisation.

UML 2.0 comporte 13 diagrammes permettant de modéliser différents aspects d'un système. On se focalisera dans ce cours sur les 9 diagrammes issus d'UML 1.5.

Diagrammes des aspects dynamiques

- 6 Diagramme de **séquences** : représente les interactions d'un point de vue chronologique ; d'une part, les interactions entre le système et les acteurs et, d'autre part, les interactions entre les objets à l'intérieur du système.
- 7 Diagramme de **collaboration** : représente les interactions entre objets d'un point de vue spatial.
- 8 Diagramme d'**états-transitions** : automates hiérarchiques permettant représenter les comportements d'un acteur, d'un système ou d'un objet d'une certaine classe.
- 9 Diagramme d'**activités** : permet de modéliser le comportement d'une méthode en représentant un enchaînement d'activités.

Les diagrammes d'UML

Un diagramme donne à l'utilisateur la possibilité de visualiser et de manipuler des éléments de modélisation.

UML 2.0 comporte 13 diagrammes permettant de modéliser différents aspects d'un système. On se focalisera dans ce cours sur les 9 diagrammes issus d'UML 1.5.

4 diagrammes UML 2.0 (non traités en cours)

- Diagramme de **paquetages** : permet de décrire les interactions entre les paquetages (paquetage = structure logique de regroupement et d'organisation des éléments du modèle UML).
- Diagramme de **structure composite** : décrit sous la forme d'une boîte blanche les relations entre les composants d'une classe.
- Diagramme **global d'interaction** : permet de décrire les enchaînements possibles entre les scénarios préalablement identifiés sous forme de diagrammes de séquences.
- Diagramme de **temps** : permet de décrire les variations d'une donnée au cours du temps.

Les aspects statiques

Les diagrammes de cas d'utilisation

Les diagrammes de classes et d'objets

Les diagrammes de composants

Les diagrammes de déploiement

Diagramme des cas d'utilisation

Les cas d'utilisation permettent de décrire le système du point de vue de l'utilisateur. Ils permettent de définir les limites du système et les relations entre le système et son environnement.

Identification des acteurs

Un acteur représente un rôle joué par une personne ou une chose qui interagit avec le système.

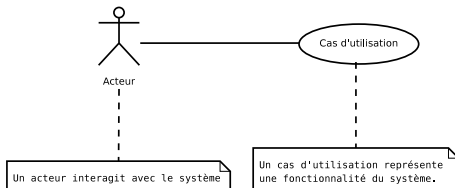
N.B. : Une même personne physique peut correspondre à plusieurs acteurs si celle-ci peut jouer plusieurs rôles.

Acteurs et cas d'utilisation

Cas d'utilisation : fonctionnalité du système utilisée par un acteur.

Un acteur qui interagit avec le système pour utiliser une fonctionnalité est représenté par une relation entre cet acteur et le cas d'utilisation représentant cette fonctionnalité.

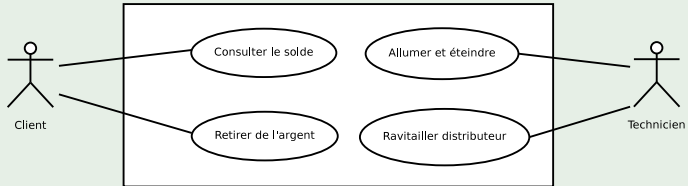
On dit que l'acteur *déclenche* le cas d'utilisation auquel il est lié.



Exemple de cas d'utilisation

Distributeur de billets

- Un premier acteur : "client", consulte le solde de son compte, ou retire de l'argent.
- Un second acteur : "technicien", allume, éteint ou ravitaille le distributeur.

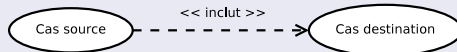


On note qu'un rectangle symbolise les limites du système.

Inclusion entre deux cas d'utilisation

Un cas d'utilisation, nommé *source*, **inclut** un cas d'utilisation nommé *destination*, si les comportements décrits par le cas source contiennent les comportements décrits par le cas destination.

Notation de l'inclusion

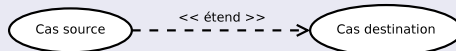


Utilisé en particulier pour factoriser une fonctionnalité partagée par plusieurs cas d'utilisation.

Extension d'un cas d'utilisation

Un cas d'utilisation, nommé *source*, **étend** un cas d'utilisation nommé *destination*, si les comportements décrits par le cas source étendent les comportements décrits par le cas destination.

Notation de l'extension

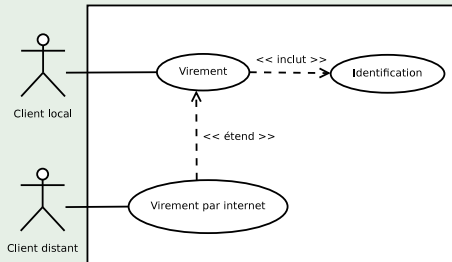


L'extension peut être soumise à une condition.

Exemple d'inclusion et d'extension

Système de virements par internet

- Client local : virements (incluant une identification)
- Client distant : virements par internet (étendant le virement standard)



Petit exercice

Enoncé

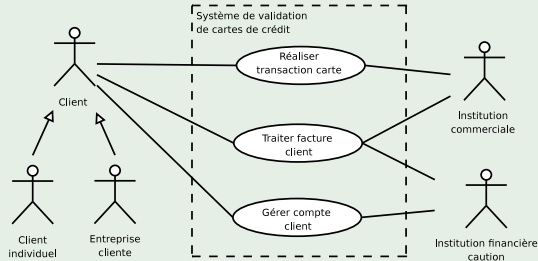
On souhaite modéliser les interactions de différents utilisateurs avec un système de validation de cartes de crédit (basiquement un terminal avec un lecteur de cartes).

Les cartes de crédit sont détenues par des clients, pouvant être soit des personnes individuelles ou une entreprise, et sont utilisées sous la supervision d'une institution. Une institution commerciale supervise les transactions et le traitement de factures des clients.

Une institution financière supervise également ces factures, mais s'occupe aussi de gérer le compte du client, avec son accord.

Petit exercice

Une solution possible



Cette solution fait apparaître un héritage entre acteurs : les comportements de chacun des deux sous-acteurs sont factorisés dans le super-acteur.

Les aspects statiques

Les diagrammes de cas d'utilisation

Les diagrammes de classes et d'objets

Les diagrammes de composants

Les diagrammes de déploiement

Les diagrammes de classes et d'objets

Le diagramme de classes

Il se présente sous la forme d'un réseau :

- de **classes**, décrivant un ensemble d'objets, et
- d'**associations**, décrivant un ensemble de liens.

Il donne les informations sur les liens d'un objet vers les autres objets.

Le but est de décrire ce qu'il faut réaliser, et non pas comment le réaliser.

Le diagramme d'objets

C'est une instanciation d'un diagramme de classe. Il se présente sous la forme d'un réseau :

- d'**objets**, instances des classes du diagramme associé, et
- de **liens**, décrivant les interactions existantes entre les objets

2.1. Les classes

La notion de classe

Définition

Une classe est une abstraction qui représente un ensemble d'objets, ayant une structure et un comportement commun.

Par exemple : "Etudiant" est une classe d'objets.

Un objet est une **instance** de sa classe ; une classe est dite **instanciée** lorsqu'elle contient au moins un objet.

Concrètement ...

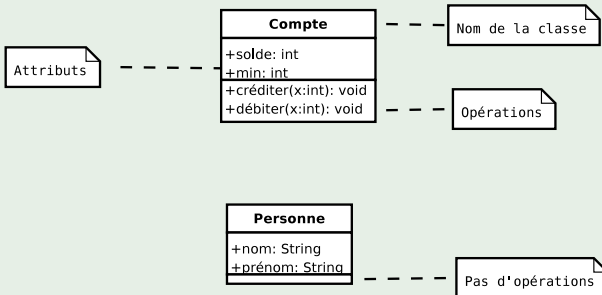
Une classe contient :

- un nom
- des attributs
- des opérations

Exemple et représentation

Classes **Compte** et **Personne**

Représentation des classes **Compte** et **Personne**



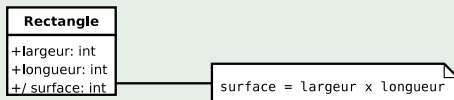
Les attributs

Un attribut est une propriété d'un objet de la classe qui comporte :

- un nom (unique dans sa classe)
- un type (facultatif, mais conseillé)
- une valeur initiale (facultative)

Un attribut est dit **dérivé** si sa valeur peut être calculée à partir d'attributs non-dérivés. Leurs noms sont précédés par '/'.
/

Attribut dérivé



Chaque attribut à une valeur pour chaque instance de classe. Les instances de classe peuvent avoir des valeurs identiques ou différentes pour un attribut donné.

Les opérations et les méthodes

Une opération est un service offert par les objets de la classe.

En UML, on distingue **opération** et **méthodes** :

- une opération spécifie un service
- une méthode implante une opération

Syntaxe d'une opération

Opération ($mode_1 \ arg_1 : Type_1 = Val_1, \dots, mode_N \ arg_N : Type_N = Val_N$) : $TypeR$

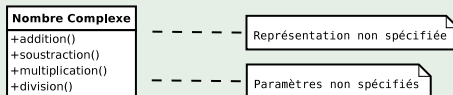
- Le *mode* peut être : in (paramètre d'entrée), out (paramètre de sortie), inout (paramètre d'entrée et de sortie). Le mode par défaut est in.
- Les paramètres, le type et la valeur initiale sont optionnels.

Visibilité des attributs et des opérations

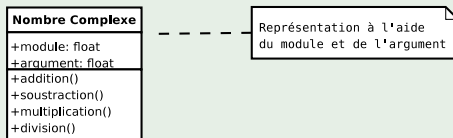
Il y a 4 niveau de visibilité qui peuvent s'appliquer aux attributs et aux opérations.

- Le niveau **public** (+), qui indique qu'un élément est visible par tous les clients de la classe.
- Le niveau **protégé** (#), qui indique qu'un élément n'est accessible que pour les sous-classes de la classe où il apparaît.
- Le niveau **paquetage** (~), qui indique qu'un élément est visible uniquement pour les classes définies dans le même paquetage.
- Le niveau **privé** (-), qui indique que seule la classe où est défini cet élément peut y accéder.

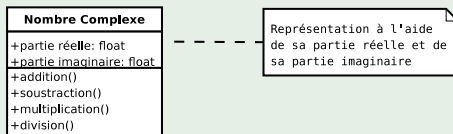
Nombres complexes, sans préciser l'implantation



Nombres complexes, implantation à l'aide du module et de l'argument



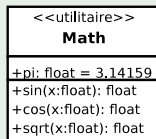
Nombres complexes, implantation à l'aide de la partie réelle et de la partie imaginaire



Classes utilitaires

Une classe utilitaire permet de regrouper un ensemble de valeurs (valeurs d'attributs) et d'opérations. Elle ne peut être instanciée et ne comporte que des attributs ou opérations de classe.

Classe utilitaire *Math*



`<<utilitaire>>` est un stéréotype indiquant que Math est utilitaire, elle ne peut donc pas être instanciée

Stéréotypes sur les classes

Une classe peut admettre un stéréotype, écrit entre chevrons au dessus du nom de la classe. Similairement, les stéréotypes peuvent s'appliquer à certaines opérations des classes.

Les stéréotypes permettent d'étendre la sémantique des éléments de modélisation. Ils permettent de définir de nouvelles classes d'éléments de modélisation, en plus du noyau prédéfini par UML. UML propose de nombreux stéréotypes "standards" mais d'autres plus spécifiques peuvent être créés pour des besoins de modélisation.

Quelques stéréotypes standards

- « thread », appliqué à une classe, qui indique que cette classe représente un processus léger
- « utility », appliqué à une classe, qui indique que cette classe est une classe utilitaire
- « constructor », « getter », « setter », appliqués à des opérations.

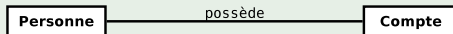
2.2. Les associations

Définition d'une association

Une association est une relation n -aire entre n classes d'objets. Les relations les plus utilisées sont les relations binaires, entre deux classes.

Exemple d'association informel

Relation de possession entre une personne et un compte courant.



- Classes : Personne et Compte
- Association : relation "possède"

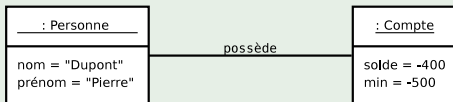
Définition d'une association

Une association est une relation n -aire entre n classes d'objets. Les relations les plus utilisées sont les relations binaires, entre deux classes.

On appelle **lien** l'instanciation d'une association entre deux objets.

Exemple de lien informel

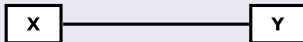
Pierre Dupont possède le compte dont le solde est à -400.



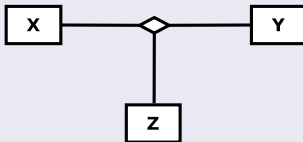
Représentation graphique d'une association

Une association compte au moins deux extrémités. Elle peut être binaire, ternaire, ou d'un autre ordre.

Représentation d'une association binaire



Représentation d'une association ternaire



Les multiplicités

La multiplicité précise combien d'instances d'une classe peuvent se rattacher à une seule instance d'une classe associée.

Valeurs de multiplicité

1 un et un seul

0..1 cas optionnel

N si on connaît la valeur, on la précise

M..N intervalle de valeurs (de $M \in \mathbb{N}$ à $N \in \mathbb{N}$)

* 0 ou plusieurs (identique à 0..*)

1..* au moins un

Graphiquement, la multiplicité se place du côté de la classe qu'elle décrit.

D'un point de vue mathématique

Les associations

Une association entre deux classes A et B est une relation binaire, au sens mathématique, c'est-à-dire un sous-ensemble de $A \times B$.

$$R \subseteq A \times B \quad (1)$$

Il est donc impossible d'avoir deux liens différents symbolisant une même association entre deux objets identiques.

Les relations

Soit R une relation entre deux classes A et B , avec les multiplicités M_A , associée à A , et M_B , associée à B . Les multiplicités M_A et M_B imposent les contraintes suivantes :

- Pour chaque objet b de la classe B , le nombre d'objets de la classe A liés à b appartient à M_A .

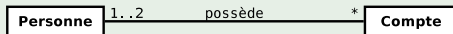
$$\forall b \in B, \text{card}\{(a, b) \in R; a \in A\} \in M_A \quad (2)$$

- Pour chaque objet a de la classe A , le nombre d'objets de la classe B liés à a appartient à M_B .

$$\forall a \in A, \text{card}\{(a, b) \in R; b \in B\} \in M_B \quad (3)$$

Un exemple : compte et personne

Diagramme de classes

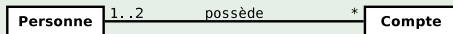


Ceci signifie :

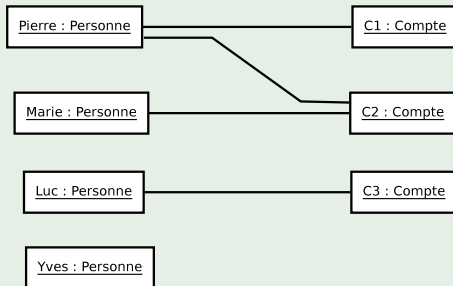
- Une personne peut ouvrir un nombre quelconque de comptes
- Un compte est ouvert pour une ou deux personnes

Un exemple : compte et personne

Diagramme de classes

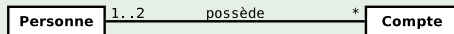


Un diagramme d'objets correct

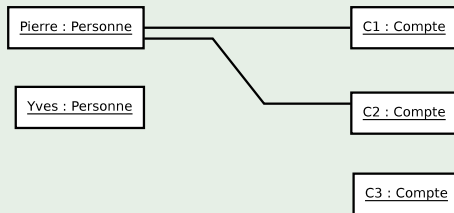


Un exemple : compte et personne

Diagramme de classes



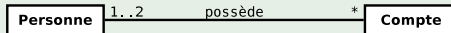
Un diagramme d'objets incorrect



Navigabilité d'une association

Par défaut, une association est navigable dans les deux sens.

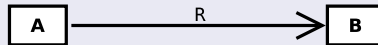
Diagramme de classes



- à partir d'une personne, on peut retrouver les comptes qui lui sont associés ;
- à partir d'un compte on peut retrouver son (ou ses deux) propriétaire(s)

Pour spécifier qu'un seul sens de navigation est autorisé, on peut orienter la relation.

Association orientée

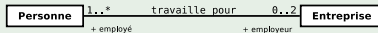


A partir d'un objet de la classe A, on peut accéder aux objets de la classe B qui lui sont liés. L'inverse n'est pas autorisé.

La notion de rôles

Les rôles désignent les extrémités d'une association. Ils peuvent être nommés.

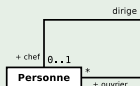
Exemple de rôles



- une personne joue le rôle d'un employé
- une entreprise joue le rôle d'un employeur

Principalement utilisés dans les associations réflexives :

Deux exemples d'associations réflexives

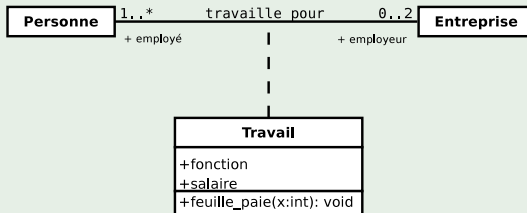


Classe-associations

Une classe-association est une entité qui combine les propriétés d'une classe et les propriétés d'une association.

Une association R entre deux classes A et B permet d'associer à tout lien entre un objet de la classe A et un objet de la classe B un objet de la classe R .

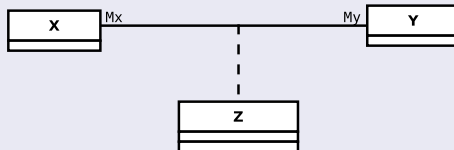
Exemple de classe-association



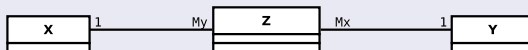
Classe-associations et multiplicités

Une classe-association entre deux classes peut se simuler par l'intermédiaire d'une classe et de deux associations.

Simulation d'une classe association

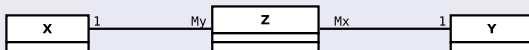


Se simule par :



Classe-associations et multiplicités

Simulation d'une classe association

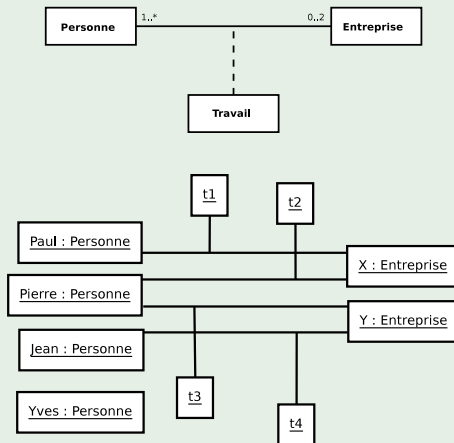


Lecture des multiplicités

- Chaque objet de la classe Z est associé à un objet de la classe X et un objet de la classe Y
- Chaque objet de la classe X est associé à n objets de la classe Z, avec $n \in M_Y$
- Chaque objet de la classe Y est associé à n objets de la classe Z, avec $n \in M_X$

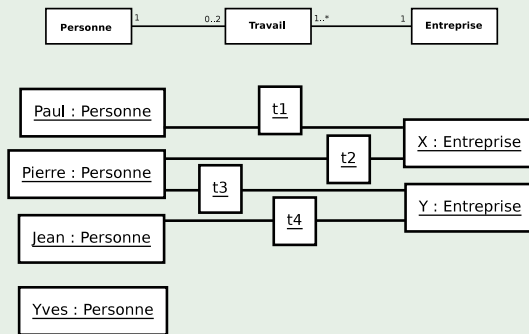
Classes-associations et objets

Classe-association



Classes-associations et objets

Classe-association simulée



2.3. Les associations plus complexes

Agrégation et composition (1/2)

L'agrégation est une association binaire particulière qui modélise une relation entre un "tout" et une "partie", autrement dit, une relation d'appartenance.

Notation de l'agrégation



Un objet de la classe B, appelé **agrégat** contient un certain nombre d'objets de la classe A, appelés **composants**.

Exemple d'agrégation

Diagramme de classes

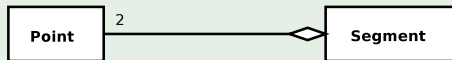
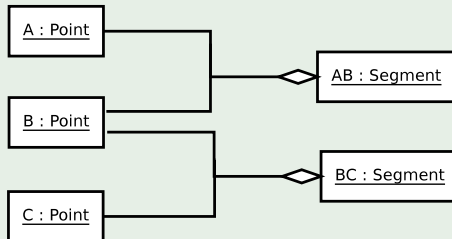


Diagramme d'objets



Remarque sur l'agrégation

La relation d'agrégation interdit les cycles dans les diagrammes d'objets.
Par contre, dans le diagramme de classes, on peut définir une classe A en agrégation avec elle-même.

Diagramme de classes et diagramme d'objet

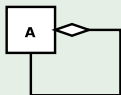


Diagramme correct

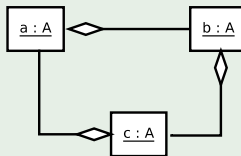


Diagramme incorrect : cycles interdits

Dans une agrégation, certaines parties peuvent être partagées. Par défaut, il n'y a pas de contrainte de multiplicité sur l'agrégat.

Agrégation et composition (2/2)

La composition est une agrégation particulière pour laquelle une partie ne peut pas être partagée entre plusieurs agrégats. Autrement dit, la multiplicité du côté de l'agrégat est 0 ou 1.

Notation de la composition, équivalence avec l'agrégation



Les cycles de vies des composants et de l'agrégat sont liés : si l'agrégat est détruit, ses composants le sont aussi.

Exemple de composition

Diagramme de classes

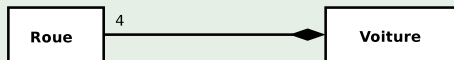
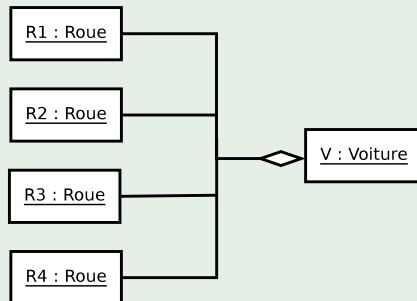


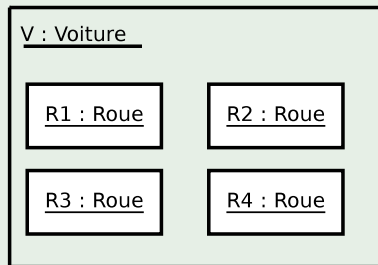
Diagramme d'objets



Remarque sur la composition

Il est également possible de représenter les composants d'un objet à l'intérieur de l'objet lui-même.

Représentation



Attention : ceci n'est possible que pour les compositions car deux objets ne peuvent pas partager un même composant.

Associations n -aires

Une association n -aire est une relation entre n classes.

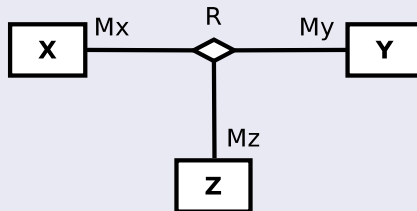
Mathématiquement parlant ...

Une association n -aire entre les classes $A_1, A_2, \dots, A_n$ est un sous-ensemble R de $A_1 \times A_2 \times \dots \times A_n$, donc un ensemble de n -uplets de la forme $(a_1, a_2, \dots, a_n)$ avec $\forall i \in 1, 2, \dots, n, a_i \in A_i$.

Associations n -aires, multiplicités

La multiplicité associée à une classe pose une contrainte sur le nombre de n -uplets de la relation lorsque les $n - 1$ autres valeurs sont fixées.

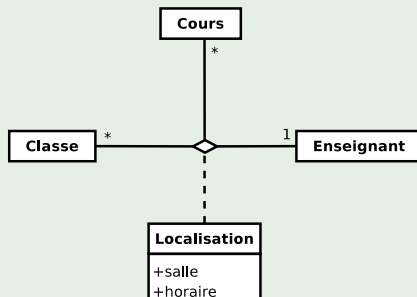
Notation



- $\forall y_0 \in Y, \forall z_0 \in Z, \text{card}((x, y_0, z_0) \in R; x \in X) \in M_X$
- $\forall x_0 \in X, \forall z_0 \in Z, \text{card}((x_0, y, z_0) \in R; y \in Y) \in M_Y$
- $\forall x_0 \in X, \forall y_0 \in Y, \text{card}((x_0, y_0, z) \in R; z \in Z) \in M_Z$

Exemple d'association n -aire

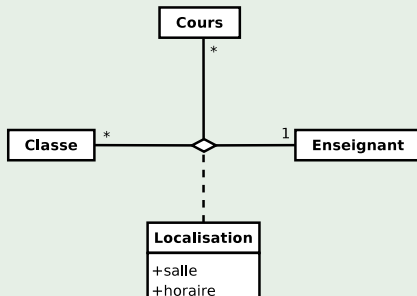
Association ternaire



- un enseignant peut faire un même cours à plusieurs classes (multiplicité “*” associée à la classe Classe)

Exemple d'association n -aire

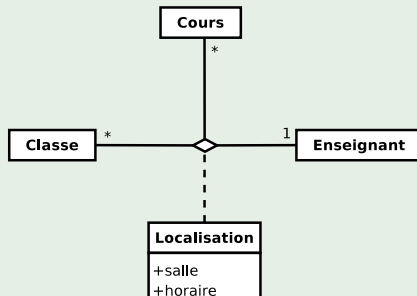
Association ternaire



- dans une classe, un cours est effectué par un seul enseignant (multiplicité "1" associée à la classe Enseignant)

Exemple d'association n -aire

Association ternaire



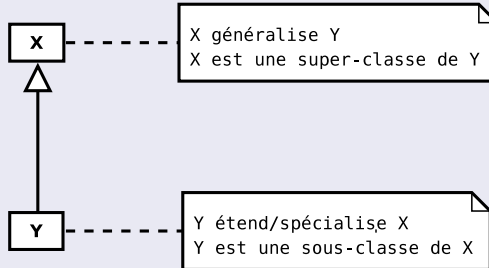
- un enseignant peut faire plusieurs cours dans une même classe (multiplicité “*” associée à la classe Cours)
Un enseignant peut donc enseigner plusieurs matières.

L'extension

Une classe Y **étend** ou **spécialise** une classe X si tout objet de la classe Y est, ou peut être considéré comme, un objet de la classe X.

A l'inverse, on dira que la classe X **généralise** la classe Y.

Notation de l'extension



Les propriétés de l'extension

Propriété d'héritage

Les attributs et les opérations définis dans X, qui ne sont pas privés, et qui ne sont *redéfinis* dans Y, sont *hérités* dans Y.

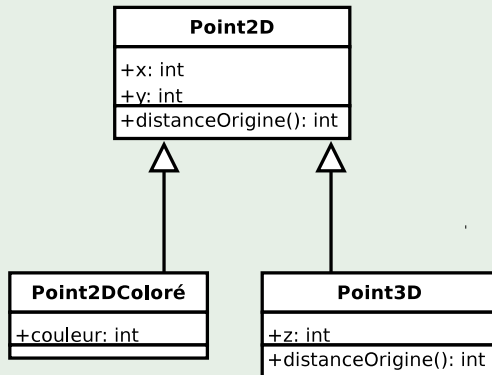
Propriété de substitution

La propriété de substitution exprime que tout objet de la classe Y *est* ou *peut être considéré comme* un objet de la classe X.

Chaque fois qu'on a besoin d'une instance de X, on peut utiliser à la place une instance de Y.

Un exemple d'extension

Exemple avec des points



Hiérarchie des classes

L'extension permet d'exprimer la notion de hiérarchie entre différentes classes.

Hiérarchie de véhicules

On considère différents types de *véhicules* :

- Les *véhicules aériens* : les *avions*
- Les *véhicules aquatiques* : les *bateaux* et les *sous-marins*
- Les *véhicules terrestres* : les *chars* et les *voitures*

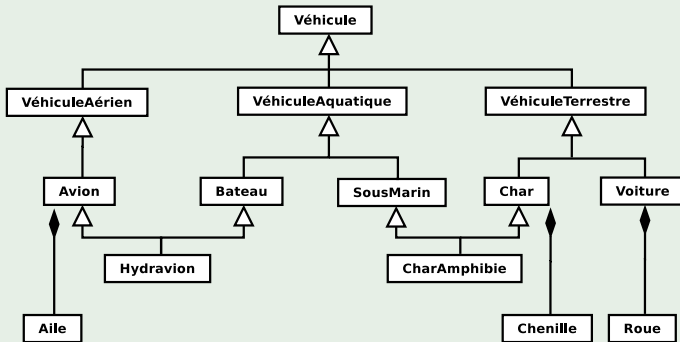
On précise :

- qu'un *hydravion* est à la fois un avion et un bateau,
- qu'un *char amphibie* est à la fois un sous-marin et un char
- qu'un avion est composé d'*ailes*
- qu'un char est composé de *chenilles*
- qu'une voiture est composée de *roues*

Hiérarchie des classes

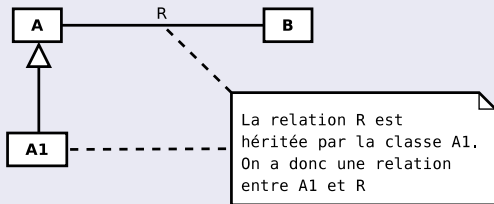
L'extension permet d'exprimer la notion de hiérarchie entre différentes classes.

Hiérarchie de véhicules (diagramme)



Héritage de relation

Relation héritée



Dans l'exemple précédent : un hydravion est, en tant qu'avion, également composé d'ailes, et un char amphibie est, en tant que char, également composé de chenilles.

Classe abstraite

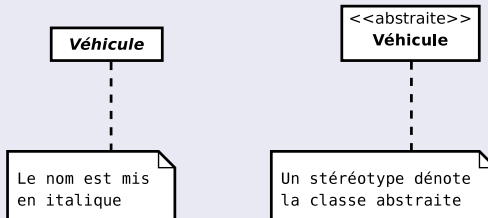
Une classe abstraite est une classe qui ne peut être instanciée, et qui peut contenir des opérations abstraites.

Une opération abstraite est une opération sans implantation, c'est-à-dire à laquelle ne correspond aucun code. Une classe abstraite peut aussi contenir des opérations concrètes (= non-abstraites).

Les classes abstraites servent notamment dans les hiérarchies de classes, où elles permettent de regrouper des attributs et des opérations communs à plusieurs classes.

En programmation objet (par exemple, en Java) une classe concrète doit implanter les opérations abstraites dont elle hérite.

Notation



Exercice

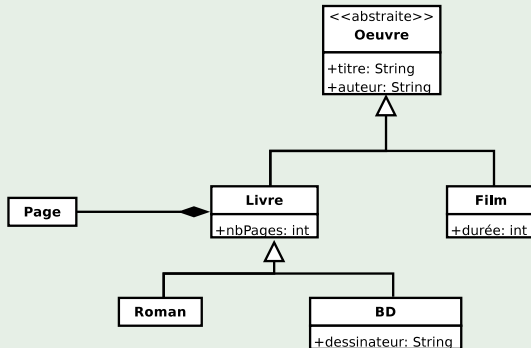
Description

On souhaite modéliser des œuvres, qui sont soit des bandes dessinées, des romans, ou des films. Ces œuvres ont un titre et un auteur. Les livres ont un certain nombre de pages, et une bande dessinée possède en plus un dessinateur. Les films, eux, ont une durée bien définie.

Réaliser le diagramme de classes correspondant.

Exercice

Une solution possible



Les interfaces

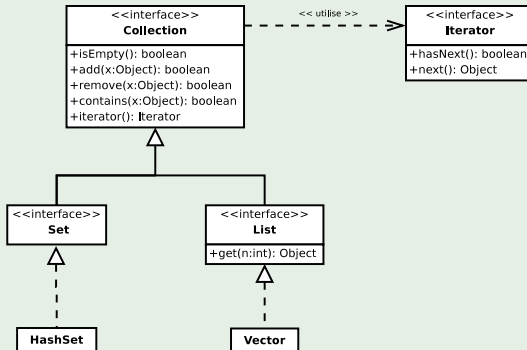
Une interface spécifie un ensemble d'opérations qui constituent un ensemble cohérent. Elle contient uniquement des opérations abstraites, sans implantation.

Une interface est équivalente à une classe abstraite qui ne contient que des opérations abstraites.

Une classe *implante* (ou *implémente* ou *réalise*) une interface lorsqu'elle fournit toutes les opérations de l'interface.

Exemple d'interfaces

Les collections en Java



On note un lien d'utilisation entre les deux interfaces, et deux liens de réalisation (HashSet implante Set et Vector implante List).

2.3. Object Constraint Language

Quelques notions d'OCL

UML formalise l'expression des contraintes avec OCL (Object Constraint Language).

- OCL est une contribution d'IBM à UML 1.1.
- Ce langage formel est volontairement simple d'accès et possède une grammaire élémentaire (OCL peut être interprété par des outils).
- OCL permet ainsi de limiter les ambiguïtés du langage naturel, tout en restant accessible.
- OCL permet de décrire des invariants dans un modèle, sous forme de pseudo-code :
 - pré et post-conditions pour une opération,
 - expressions de navigation,
 - expressions booléennes, etc...

OCL permet d'enrichir ce diagramme, en décrivant toutes les contraintes et tous les invariants du modèle présenté, de manière normalisée et explicite (à l'intérieur d'une note rattachée à un élément de modélisation du diagramme).

Syntaxe des contraintes OCL (1/5)

Contexte d'une contrainte

Une contrainte s'applique sur tous les objets de la classe considérée.

Le contexte définit la portée de la contrainte. Il peut s'agir :

- d'une classe : la contrainte devra alors être un invariant, précisé par `inv`
context *MaClasse*
 `inv` : ...
- d'une opération : la contrainte pourra alors être une précondition (`pre`) ou une postcondition (`post`).
context *MaClasse* :: *MonOperation*(*i* : *Integer*)
 `pre` : ...
 `post` : ...

Les découpages sont possible à l'intérieur d'un même contexte. La conjonction des contraintes proposées sera alors considérée.

Syntaxe des contraintes OCL (2/5)

Les expressions

Les invariants, préconditions et postconditions expriment des contraintes qui sont des prédicats de logique du premier ordre portant sur :

- l'instance courante de la classe, `self`
- les attributs d'une instance, `instance.att` (`self.att` peut s'écrire `att`)
- les collections d'objets associés aux classes `instance.role`
- les valeurs à l'état avant l'opération dans les postconditions uniquement, `attpre`
- les paramètres d'entrée ou de sortie des opérations, pour les pré- et postconditions
- la valeur de retour d'une opération, `result`

Syntaxe des contraintes OCL (2/5)

Les expressions

Quelques opérateurs de manipulation des données :

- entiers, réels : `=`, `<>`, `<=`, `>=`, `+`, `-`, `*`, `/`, `x.div(y)`, `x.mod(y)`
- chaînes : `s.concat(t)`, `s.size()`, `s.subString(debut,fin)`
- booléens : `and`, `or`, `not`, `xor`, `=`, `implies`, `if ... then ... else ... endif`
- objets : `obj.isOclUndefined()` (eq. à `obj = null`),
`obj.isOclInstance(Classe)`, `obj.isOclNew()`

Syntaxe des contraintes OCL (3/5)

Collections et accès aux objets associés d'une classe

On compte quatre type de collections :

- Set : pas de doublons, pas d'ordre
- Bag : doublons possibles, pas d'ordre
- OrderedSet : pas de doublons, ordonné
- Sequence : doublons possibles, pas d'ordre

L'expression `instance.role` renverra différents types en fonction des multiplicités du côté du role :

- multiplicité 0 ou 1, `instance.role` est un objet (éventuellement la valeur `null` si aucun objet n'est associé à `instance`)
- multiplicité N (> 1) ou M..N, `instance.role` est un Set.

Syntaxe des contraintes OCL (4/5)

Opérations de manipulation des collections

Ces opérations s'appliquent en utilisant l'opérateur `->` suite au nom de la collection.
Quelques opérations utiles :

- `coll->size` : compte le nombre d'objets d'une collection (renvoie un entier)
- `coll->isEmpty` : équivalent à `coll->size = 0`
- `coll->notEmpty` : équivalent à `coll->size <> 0`
- `coll->count(obj)` : compte le nombre d'occurrences de `obj` dans `coll`
- `coll->includes(obj)` : équivalent à `coll->count(obj) > 0`
- `coll->excludes(obj)` : équivalent à `coll->count(obj) = 0`
- `coll->includesAll(coll2)` : vérifie si tous les éléments de `coll2` sont inclus dans `coll`

Syntaxe des contraintes OCL (5/5)

Opérations collectives et simplification d'écriture

Ces opérations permettent d'effectuer des requêtes sur les collections.

Quelques opérations utiles :

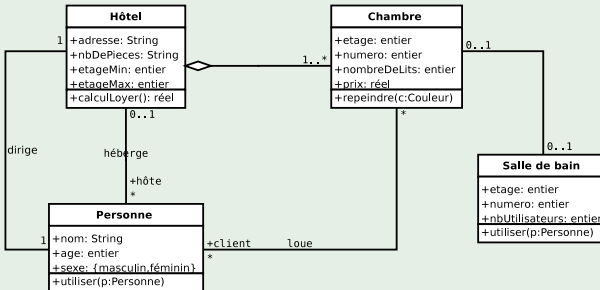
- `coll->collect(role)` : renvoie un Bag contenant les objets liés à ceux de `coll` par le rôle `role`
- `coll->exists(expr)` : vérifie si `expr` est satisfait pour au moins un élément de la collection
- `coll->forall(expr)` : vérifie si `expr` est satisfait pour tous les éléments de la collection
- `coll->select(expr)` : renvoie la collection composée des éléments de `coll` qui satisfont `expr`

Il est possible de définir des attributs ou des opérations temporaires afin de simplifier et compacter les expressions. context *MaClasse*

```
inv : let val1 :Boolean = self.att1 >= 1 and self.att1 <= 140  
      let majeur(age : Integer) :Boolean = age >= 18 in  
      val1 and (self.attr2 = 42 or majeur(self.attr3))
```

Quelques exemples de spécifications en OCL

Diagramme de classes d'un hôtel



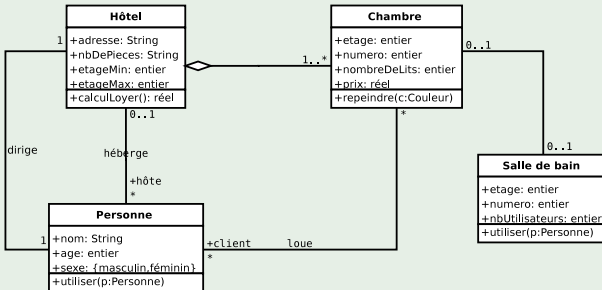
Un hôtel ne contient jamais d'étage numéro 13 (superstition oblige)

```
context Chambre inv :
self.etage <> 13
```

```
context SalleDeBain inv :
self.etage <> 13
```

Quelques exemples de spécifications en OCL

Diagramme de classes d'un hôtel

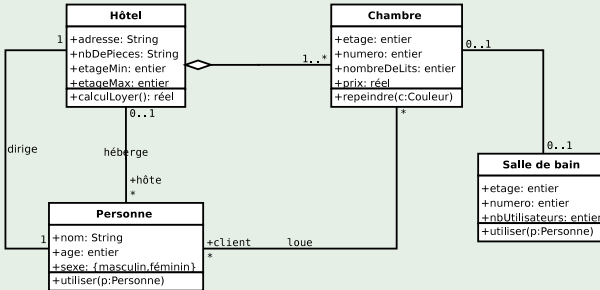


L'étage de chaque chambre est compris entre le premier et le dernier étage de l'hôtel.

```
context Hôtel inv :
self.chambre->forall(c : Chambre | c.etage <= self.etageMax and
c.etage >= self.etageMin)
```

Quelques exemples de spécifications en OCL

Diagramme de classes d'un hôtel

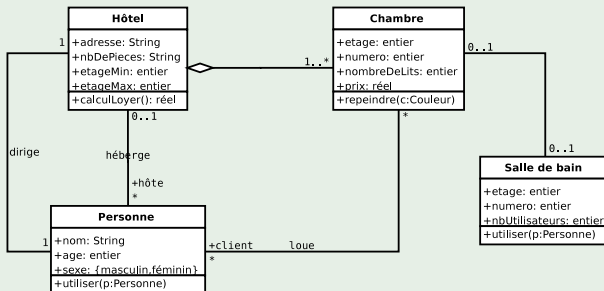


On ne peut repeindre une chambre que si elle n'est pas louée. Une fois repeinte, une chambre coûte 10% de plus.

```
context Chambre : :repeindre(c : Couleur)
pre : client->isEmpty
post : prix = prixpre * 1.1
```

Quelques exemples de spécifications en OCL

Diagramme de classes d'un hôtel



Le loyer de l'hôtel est égal à la somme du prix de toutes les chambres louées.

```

context Hôtel : : calculerLoyer() : réel
pre :
post : result = self.chambre->select(client->notEmpty).prix->sum
    
```

Les aspects statiques

Les diagrammes de cas d'utilisation

Les diagrammes de classes et d'objets

Les diagrammes de composants

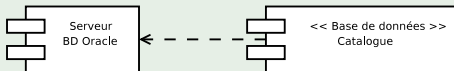
Les diagrammes de déploiement

Les diagrammes de composants

Les diagrammes de composants permettent de décrire l'architecture physique d'une application en termes de modules.

Diagramme de composants

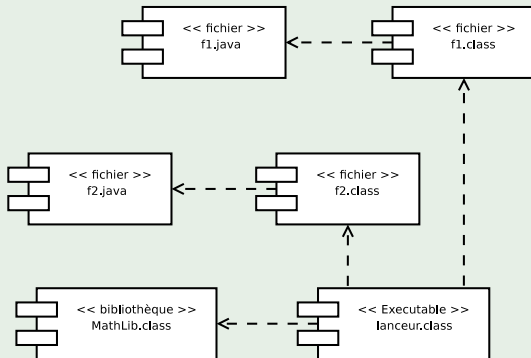
Composants d'un serveur Oracle utilisant une base de données servant de catalogue.



Diagrammes de composants

Composants

Application construite à partir de deux fichiers source et d'une bibliothèque mathématiques.



Les aspects statiques

Les diagrammes de cas d'utilisation

Les diagrammes de classes et d'objets

Les diagrammes de composants

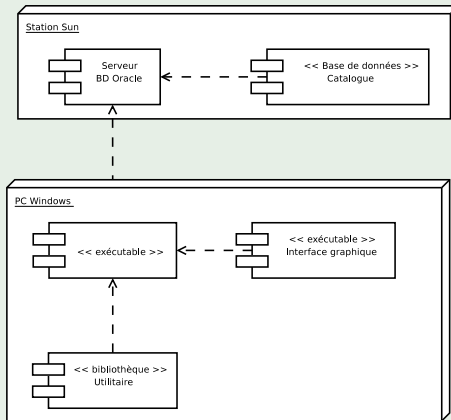
Les diagrammes de déploiement

Les diagrammes de déploiement

Les diagrammes de déploiement décrivent la disposition physique des matériels et la répartition des composants sur ces matériels.

Les diagrammes de déploiement

Diagramme de déploiement



Les aspects dynamiques

Les diagrammes de séquence

Les diagrammes de collaboration

Les diagrammes d'états-transitions

Les diagrammes d'activité

Les diagrammes de séquence

Les diagrammes de séquence permettent de représenter les *interactions* entre objets d'un point de vue chronologique.

Ils se concentrent sur la séquence des interactions d'un point de vue temporel.

Ils font partie des diagrammes dits d'interaction, avec les diagrammes de collaboration.

Les diagrammes de séquence

Les interactions se traduisent par des envois de messages entre les objets.

Les diagrammes de séquences peuvent s'appliquer

- à la documentation des cas d'utilisation, en représentant les interactions entre les acteurs et le système. Les étiquettes des messages sont des événements qui se produisent dans le système.
- à la représentation des interactions informatiques et la répartition des flots de contrôle. Les messages échangés unifient les différentes formes de communication entre les objets (appels de procédure, émission de signaux, etc.).

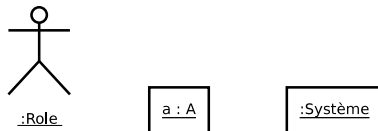
Les objets des diagrammes de séquence

Ce sont des entités appartenant au système (instance d'une classe), ou se trouvant à ses limites (les acteurs), ou le système lui-même.

Ils représentent :

- Soit des concepts abstraits, ou des acteurs (documentation des cas d'utilisations)
- Soit des objets d'implantation (interactions informatiques)

Les objets sont identifiés par l'intermédiaire des diagrammes de cas d'utilisation ou des diagrammes de classes. Ils sont dénotés classiquement.

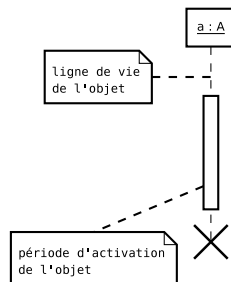


La ligne de vie des objets

Elle représente la période de temps durant laquelle l'objet existe.

La période d'activation correspond à un intervalle de temps durant lequel l'objet effectue une actions, soit directement, soit par l'intermédiaire d'un autre objet qui lui sert de sous-traitant (appel de procédure).

La vie de l'objet commence par l'objet lui-même et se termine à la fin de la ligne de vie, éventuellement marquée par une croix indiquant la destruction de l'objet.



Les types de messages

On distingue 3 types de messages.



1. Message synchrone (appel de procédure)



2. Message asynchrone



3. Retour de procédure

1. Message synchrone : correspond à un appel de procédure ou flot de contrôle imbriqué. La séquence imbriquée est entièrement faite avant que l'appelant ne continue : l'appelant est bloqué jusqu'à ce que l'appelé termine.

Les types de messages

On distingue 3 types de messages.



1. Message synchrone (appel de procédure)



2. Message asynchrone



3. Retour de procédure

2. Message asynchrone : correspond à un message "à plat", non imbriqué. L'appelant n'est pas bloqué jusqu'à ce que l'appelé termine.

Les types de messages

On distingue 3 types de messages.



1. Message synchrone (appel de procédure)



2. Message asynchrone



3. Retour de procédure

3. Retour de procédure : correspond à la terminaison d'une procédure avec le retour éventuel d'une valeur. Ces messages peuvent ne pas être dessinés, ils sont alors implicites.

Un premier exemple

Exemple d'un système de communication téléphonique

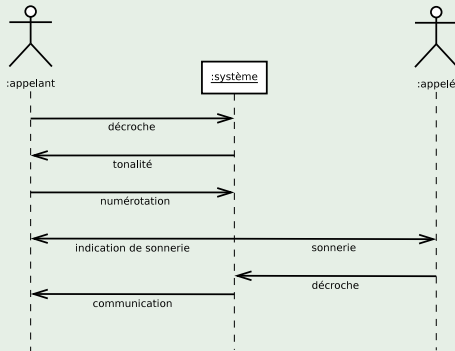
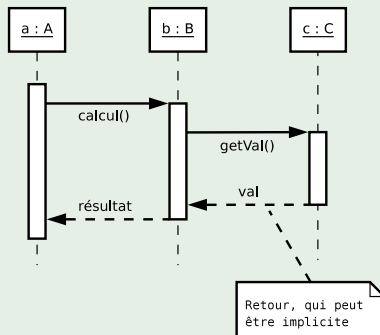


Illustration d'un diagramme de cas d'utilisation, avec messages asynchrones.

Un second exemple

Contrôle de flot d'exécution

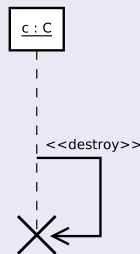
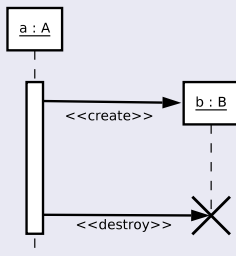


Création et destruction d'un objet

Créer et détruire des objets

Deux types de messages particuliers permettent de créer et de détruire des objets.

- La création se symbolise par un message `<< create >>`
- La destruction se symbolise par un message `<< destroy >>`



Étiquettes des messages

Syntaxe générale

La syntaxe des étiquettes de message est la suivante :

[Garde] Iteration resultat := nom_message(arguments)

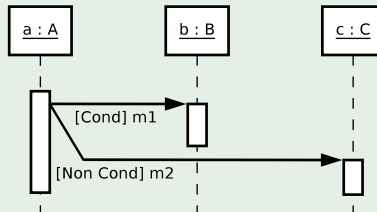
où

- *[Garde]* est la condition d'envoi du message (facultatif)
- *Iteration* décrit une éventuelle itération sur l'envoi du message (facultatif)
- *resultat :=* est l'affectation d'une valeur de retour (facultatif)
- *nom_message* est le nom du message
- *arguments* sont les arguments du message (facultatif)

Etiquettes : messages gardés

Les gardes permettent de spécifier des conditions pour l'envoi des messages. Les messages ne sont envoyés que si la condition est satisfaite.

Messages conditionnels

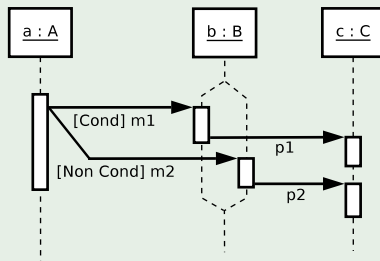


Etiquettes : messages gardés

Les gardes permettent de spécifier des conditions pour l'envoi des messages. Les messages ne sont envoyés que si la condition est satisfaite.

La ligne de vie d'un objet peut être dédoublée pour indiquer des actions qui sont effectuées suite à un message conditionnel (UML 1.5).

Dédoublage de la ligne de vie

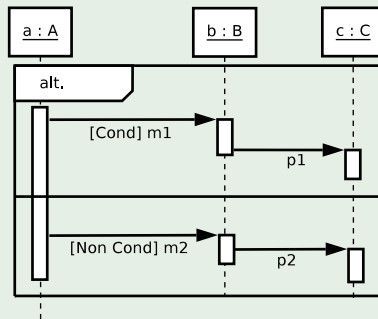


Etiquettes : messages gardés

Les gardes permettent de spécifier des conditions pour l'envoi des messages. Les messages ne sont envoyés que si la condition est satisfaite.

UML 2.0 ajoute la possibilité d'utiliser des fragments, plus lisibles.

Utilisation d'un fragment d'alternative



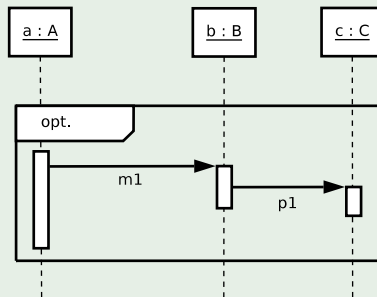
Étiquettes : messages gardés

Les gardes permettent de spécifier des conditions pour l'envoi des messages. Les messages ne sont envoyés que si la condition est satisfaite.

UML 2.0 ajoute la possibilité d'utiliser des fragments, plus lisibles.

Notamment pour les actions optionnelles.

Utilisation d'un fragment optionnel



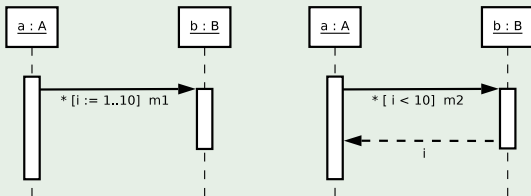
Etiquettes : itérations

Deux types d'itérations

On distingue deux types d'itérations :

- Itération *séquentielle* : envoi séquentiel de plusieurs instances du même message, dénoté par $*[clause\ d'itération]$
- Itération *parallèle* : envoi en parallèle de plusieurs instances du même message, dénoté par $* || [clause\ d'itération]$

Envoi séquentiel



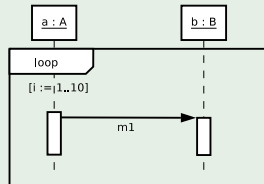
Etiquettes : itérations

Deux types d'itérations

On distingue deux types d'itérations :

- Itération *séquentielle* : envoi séquentiel de plusieurs instances du même message, dénoté par $*[clause\ d'itération]$
- Itération *parallèle* : envoi en parallèle de plusieurs instances du même message, dénoté par $* || [clause\ d'itération]$

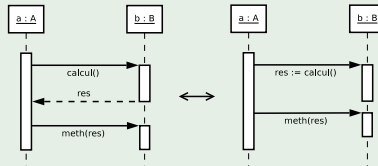
Envoi séquentiel, avec un fragment



Etiquettes : valeur de retour

Le résultat est constitué d'une liste de valeurs retournées par le message.
Ces valeurs peuvent être utilisées comme paramètres d'autres messages.

Deux notations possibles

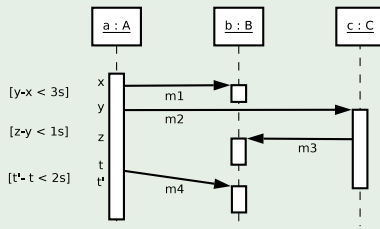


Contraintes temporelles

On peut nommer l'instant d'émission d'un message, ainsi que l'instant de sa réception. Cela permet de poser des contraintes de temps sur l'envoi et sur la réception de messages.

Par convention, si l'instant d'émission est dénoté par x , l'instant de réception de ce même message est dénoté par x' .

Utilisation de contraintes temporelles



Objets actifs

Définition

Un objet actif est un objet qui a son propre flot d'exécution.

Quelques exemples d'objets actifs

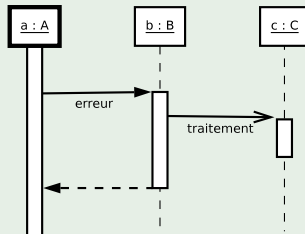
- les processus et les threads
- les objets sources d'événements, comme les boutons dans les interfaces graphiques
- les minuteries, comme les timers en Java

Ces objets permettent d'exécuter une opération soit une seule fois, après un certain intervalle de temps, soit de façon répétée à intervalles de temps donnés.

Objets actifs (suite)

Un objet actif peut, le temps d'une opération, activer un objet passif qui peut alors activer d'autres objets passifs. Lorsque l'opération est terminée l'objet passif redonne le contrôle à l'objet qui l'a activé.

Illustration



Un objet actif est représenté par des contours plus épais.

Dans un environnement multi-tâches, plusieurs objets peuvent être actifs simultanément.

Les aspects dynamiques

Les diagrammes de séquence

Les diagrammes de collaboration

Les diagrammes d'états-transitions

Les diagrammes d'activité

Les diagrammes de collaboration

Les diagrammes de collaboration permettent de représenter les *interactions* entre des objets (et éventuellement des acteurs).

Par opposition aux diagrammes de séquences, les contextes sont explicitement représentés, en précisant les états des objets qui interagissent. Pour mettre en évidence la dimension temporelle, les messages envoyés par les différents objets peuvent être numérotés.

Ils font partie des diagrammes dits d'interaction, avec les diagrammes de séquence. Le passage d'un diagramme de collaboration à un diagramme de séquence (et inversement) est simple.

Les collaborations

Les collaborations définissent les éléments utiles pour obtenir un résultat en spécifiant leurs *rôles* dans le contexte de la collaboration.

Une collaboration est composée de deux descriptions.

Description générale au niveau de la spécification

Celle-ci représente les *rôles* des cas d'utilisation, des classes, des méthodes et des associations. Une interaction sera alors une séquence de messages échangés entre les rôles.

Description spécifique au niveau instance

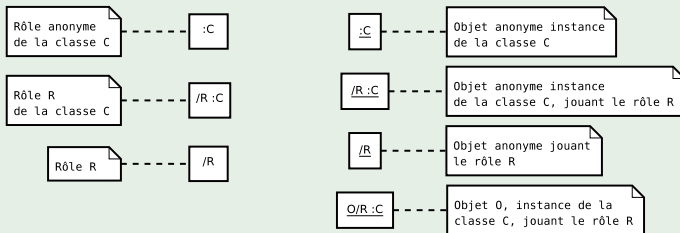
Celle-ci représente une instance particulière d'une interaction composée d'objets et de liens respectants les rôles, et de *stimulus* qui sont des instances de messages échangés entre ces objets.

Les rôles

Chaque élément d'une collaboration joue un rôle.

Chacun des classificateurs (classe, cas d'utilisation, etc.) est représenté par un symbole de classe.

Notation des rôles

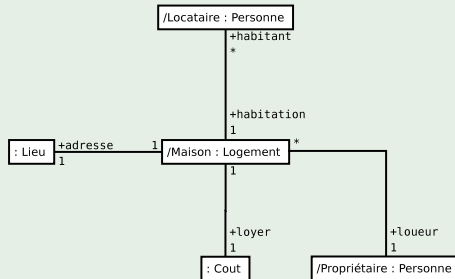


Les rôles des associations sont représentés classiquement par rapport aux diagrammes de classes.

Collaboration au niveau spécification

La collaboration forme un graphe de rôles liés par des rôles d'associations.
En général, une collaboration au niveau spécification représente un *contexte*.

Un contexte

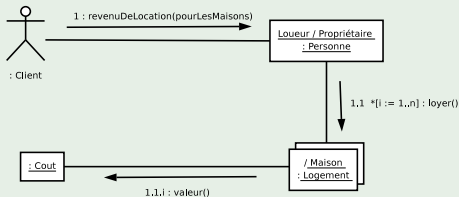


Collaboration au niveau instance

La collaboration niveau instance est un diagramme représentant une instance au niveau spécification avec des stimulus.

Un *stimulus* est une instance d'un message envoyé d'un objet vers un autre.

Exemple illustratif

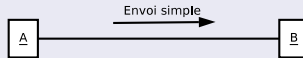


Les messages

Les objets communiquent en échangeant des messages représentés sous forme de flèche. Les messages sont étiquetés par le nom de l'opération ou du signal invoqué. L'envoi d'un message nécessite que le récepteur puisse réaliser l'opération.

Trois types de messages

On distingue 3 types de messages :



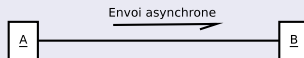
L'**appel de procédure** ou flot de contrôle emboîté : la séquence emboîtée doit se terminer pour que la séquence englobante reprenne le contrôle.

Les messages

Les objets communiquent en échangeant des messages représentés sous forme de flèche. Les messages sont étiquetés par le nom de l'opération ou du signal invoqué. L'envoi d'un message nécessite que le récepteur puisse réaliser l'opération.

Trois types de messages

On distingue 3 types de messages :



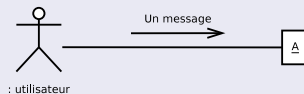
Le **flot de contrôle** asynchrone : pas de messages englobant, il n'y a donc pas d'obligation de terminaison d'autres messages pour poursuivre la séquence.

Les messages

Les objets communiquent en échangeant des messages représentés sous forme de flèche. Les messages sont étiquetés par le nom de l'opération ou du signal invoqué. L'envoi d'un message nécessite que le récepteur puisse réaliser l'opération.

Trois types de messages

On distingue 3 types de messages :



Le **flot de contrôle à plat** : cas particulier des messages asynchrones, il modélise une progression non procédurale souvent utilisée pour les messages entre un acteur et le système.

Étiquettes des messages

Syntaxe des messages

La syntaxe des étiquettes de message est la suivante :

Synchronisation [*Garde*] *Iteration* *resultat* := nom_message(*arguments*)

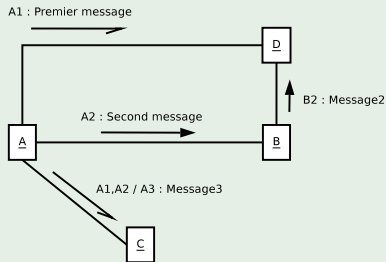
où

- *Synchronisation* permet de synchroniser le message avec d'autres (facultatif)
- [*Garde*] est la condition d'envoi du message (facultatif)
- *Iteration* décrit une éventuelle itération sur l'envoi du message (facultatif)
- *resultat* := est l'affectation d'une valeur de retour (facultatif)
- nom_message est le nom du message
- *arguments* sont les arguments du message (facultatif)

Synchronisation des messages

La synchronisation de messages permet d'envoyer un message si et seulement si d'autres messages ont déjà été envoyés.
Ceci se dénote par : $M1, M2, \dots / M$ où $M1, M2$ sont les messages dont dépend M .

Synchronisation

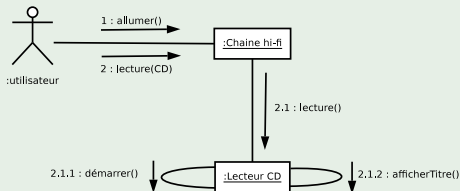


Séquence

On préfixe le message par une numérotation de manière à indiquer l'ordre d'envoi des messages.

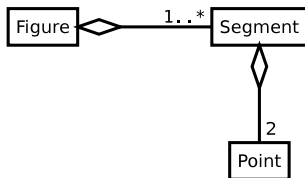
La numérotation est englobante dans le cas des appels de procédure : N (appel initial), $N.1$ (premier appel imbriqué), $N.2$ (deuxième appel imbriqué, $N + 1$ (au même niveau que N)).

Séquence de messages



Exercice

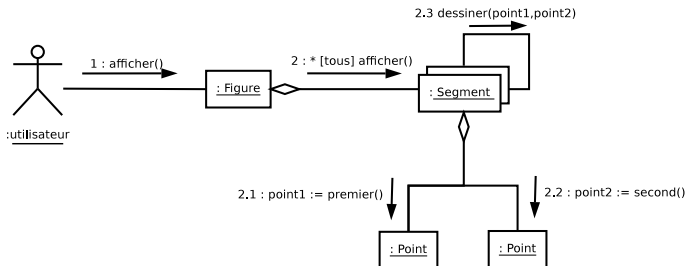
Soit le diagramme de classes suivant :



Donner les diagrammes de collaboration représentant l'affichage d'une figure demandé par un utilisateur.

Exercice - Solution

Diagramme de séquence représentant l'affichage d'une figure demandé par un utilisateur.



Les aspects dynamiques

Les diagrammes de séquence

Les diagrammes de collaboration

Les diagrammes d'états-transitions

Les diagrammes d'activité

Les diagrammes d'états-transitions

Les diagrammes d'états-transitions permettent principalement de décrire le comportement des objets d'une classe.

Ils peuvent également décrire les aspects dynamique d'un cas d'utilisation, d'un acteur, d'un système, ou d'un sous-système.

Les diagrammes d'états-transitions UML sont inspirés des *Statecharts* de David Harel. Il s'agit d'automates hiérarchiques, qui peuvent être mis en parallèle.

Les états

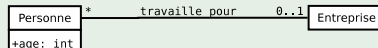
Etat d'un objet

L'état d'un objet correspond à l'ensemble des valeurs de ses attributs et à l'ensemble des liens qu'il entretient avec d'autres objets.

Un état est une condition ou une situation, dans la vie d'un objet, qui dure un certain temps pendant lequel cet objet satisfait une condition, effectue une activité, ou attend un événement.

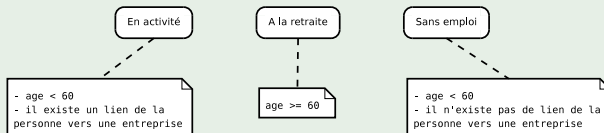
Les états : un premier exemple

Personnes employées dans une entreprise



Une personne peut avoir un des trois états suivants :

en activité, à la retraite, sans emploi



Les états : état initial et état final

Etat initial

Un *état initial* est un pseudo-état qui permet de montrer l'état dans lequel un objet se trouve au moment de sa création.



Etat initial

Etat final

Un *état final* est un pseudo-état qui permet de montrer la fin du comportement d'un objet, en particulier au moment de sa destruction.



Etat final

Transitions

Les transitions permettent à un objet de changer d'état, en fonction des événements qu'il reçoit.

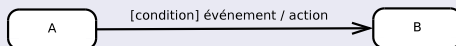
Contenu d'une transition

Une transition comporte :

- un état source
- un état destination
- un événement
- une condition, appelée garde
- une action

Transitions (suite)

Notation d'une transition



Sémantique

Si :

l'objet est dans l'état A **et** l'événement se produit **et** la condition est vraie
alors l'objet effectue l'action et passe dans l'état B.

Si aucune transition partant de l'état courant n'est étiquetée par l'événement reçu,
alors rien ne se passe ; en particulier, l'objet ne change pas d'état.

Remarque : Le passage d'un état à un autre est considéré comme instantané.

Evénements

On distingue 4 sortes d'événements en UML :

- Un *événement d'appel* ("call event") est un événement causé par l'appel d'une opération. Dans ce cas, l'événement est de la forme $op(x_1, x_2, \dots, x_N)$ où op est une opération de la classe.

Événements

On distingue 4 sortes d'événements en UML :

- Un *événement d'appel* ("call event") est un événement causé par l'appel d'une opération. Dans ce cas, l'événement est de la forme $op(x_1, x_2, \dots, x_N)$ où op est une opération de la classe.
- Un *événement modification* ("change event") est un événement causé par le passage d'une condition de la valeur faux à la valeur vrai, suite à un changement de valeur d'un attribut ou d'un lien.

Evénements

On distingue 4 sortes d'événements en UML :

- Un *événement d'appel* ("call event") est un événement causé par l'appel d'une opération. Dans ce cas, l'événement est de la forme $op(x_1, x_2, \dots, x_N)$ où op est une opération de la classe.
- Un *événement modification* ("change event") est un événement causé par le passage d'une condition de la valeur faux à la valeur vrai, suite à un changement de valeur d'un attribut ou d'un lien.
- Un *événement temporel* ("time event") est un événement qui survient quand une temporisation arrive à expiration. Une temporisation peut être relative (délai), ou absolue (spécification de l'heure à laquelle la transition doit être effectuée).

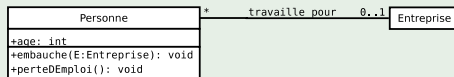
Événements

On distingue 4 sortes d'événements en UML :

- Un *événement d'appel* ("call event") est un événement causé par l'appel d'une opération. Dans ce cas, l'événement est de la forme $op(x_1, x_2, \dots, x_N)$ où op est une opération de la classe.
- Un *événement modification* ("change event") est un événement causé par le passage d'une condition de la valeur faux à la valeur vrai, suite à un changement de valeur d'un attribut ou d'un lien.
- Un *événement temporel* ("time event") est un événement qui survient quand une temporisation arrive à expiration. Une temporisation peut être relative (délai), ou absolue (spécification de l'heure à laquelle la transition doit être effectuée).
- Un *événement signal* ("signal event") est un stimulus asynchrone entre deux objets. Par exemple, un clic de souris est un signal.

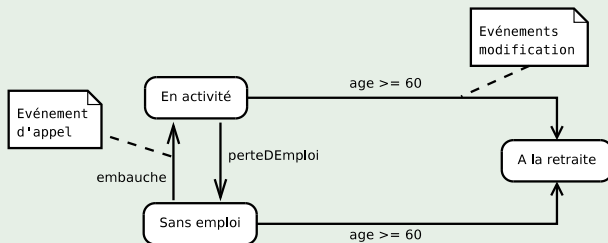
Exemple 1 : activité d'une personne

Diagramme de classe



Exemple 1 : activité d'une personne

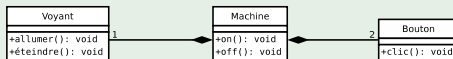
Diagramme d'états-transitions associé à la classe "Personne"



Exemple 2 : deux boutons et 1 voyant

On considère une machine avec deux boutons permettant de mettre une machine sous tension (signal "on") ou de l'éteindre (signal "off"). Le voyant indique si la machine est sous tension ou hors tension. Après une minute sans utilisation, la machine se met automatiquement hors tension.

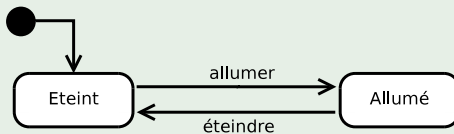
Diagramme de classe



Exemple 2 : deux boutons et 1 voyant

On considère une machine avec deux boutons permettant de mettre une machine sous tension (signal "on") ou de l'éteindre (signal "off"). Le voyant indique si la machine est sous tension ou hors tension. Après une minute sans utilisation, la machine se met automatiquement hors tension.

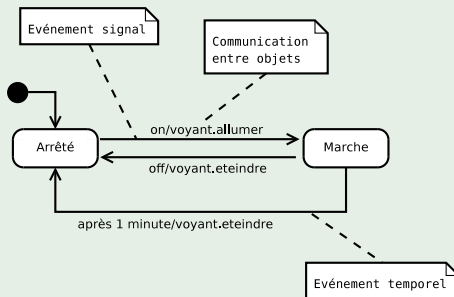
Diagramme d'états-transitions associé à la classe "Voyant"



Exemple 2 : deux boutons et 1 voyant

On considère une machine avec deux boutons permettant de mettre une machine sous tension (signal "on") ou de l'éteindre (signal "off"). Le voyant indique si la machine est sous tension ou hors tension. Après une minute sans utilisation, la machine se met automatiquement hors tension.

Diagramme d'états-transitions associé à la classe "Machine"



Remarque

Les automates considérés en UML sont *a priori* non-déterministes.

On peut donc avoir deux transitions étiquetées par le même événement qui partent du même état.

Néanmoins, d'un point de vue méthodologique, il est souvent préférable d'utiliser des **automates déterministes** pour plus de clarté.

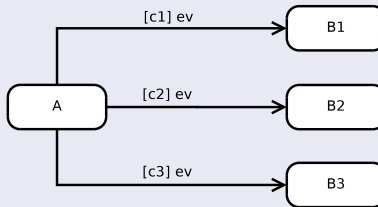
Gardes

Une garde est une condition booléenne, notée entre crochets [].

La garde de la transition est évaluée quand un événement se produit.

Gardes et déterminisme

Supposons que plusieurs transitions partant du même état A soient déclenchées par le même événement.



Pour que l'automate soit déterministe, il faut que les gardes c1, c2 et c3 soient mutuellement exclusives.

Gardes (suite)

Sémantique

Si aucune condition n'est vérifiée, alors rien ne se passe et l'objet ne change pas d'état.

Remarque : Il ne faut pas confondre un événement de déclenchement et une garde :

- Un événement de changement est un événement qui déclenche une transition lorsque la condition passe à vrai ;
- Une garde est une condition booléenne évaluée lorsque l'événement se produit.

Action et activité

Action

Une action consiste en la génération d'un signal ou l'invocation d'une opération. Une action est considérée comme *instantanée* (ie. le temps d'exécution est négligeable) et *atomique* (ie. non interruptible).

Activité

Une activité correspond à une opération qui prend un temps non négligeable et peut être interrompue.

Actions associées aux états

Les actions sont généralement associées aux transitions, mais on peut également les associer aux états. On peut en particulier :

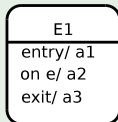
- spécifier une action à effectuer lorsqu'on entre dans un état :
entry/ action
- spécifier une action à effectuer si un événement survient :
on événement / action (l'événement est "interne")
- spécifier une action à effectuer lorsqu'on sort d'un état :
exit / action
- spécifier une activité à effectuer lorsqu'on est dans l'état :
do/ activité

Actions associées aux états (suite)

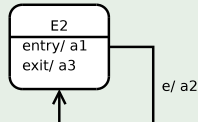
Remarque : le déclenchement d'un événement interne n'entraîne pas l'exécution des actions d'entrée et de sortie.

Illustration

On suppose qu'on entre dans l'un des deux états E1 ou E2 et qu'on reçoit une succession d'événements e.



a1 a2* a3



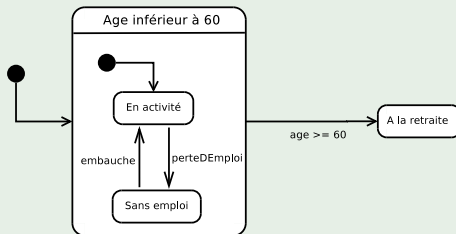
a1 (a3 a2 a1)* a3

Etats composites

Définition

Un état composite est un état qui se décompose en plusieurs sous états. Les états composites permettent de structurer les automates pour les rendre plus lisibles. Ils permettent en particulier de factoriser des transitions similaires qui partent de plusieurs états.

Activités d'une personne



Etats composites

Définition

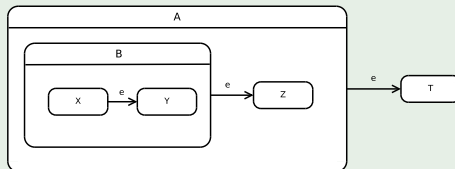
Un état composite est un état qui se décompose en plusieurs sous états. Les états composites permettent de structurer les automates pour les rendre plus lisibles. Ils permettent en particulier de factoriser des transitions similaires qui partent de plusieurs états.

Remarque : Il peut exister des transitions entre différents niveaux de l'automate, mais il est préférable de limiter leur nombre.

Etats composites (suite)

Lorsque deux transitions peuvent être effectuées, l'une sur un sous-état, et l'autre sur l'état englobant, c'est la transition sur le sous-état (la plus "spécialisée") qui est effectuée.

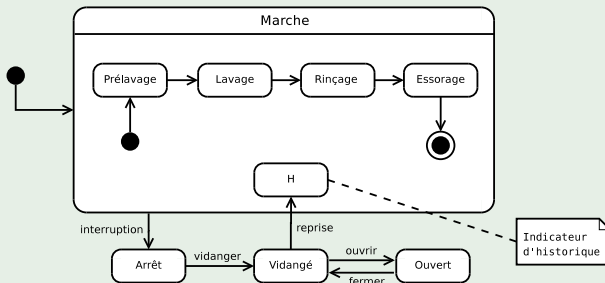
Priorité des transitions



Indicateurs d'historique

L'indicateur d'historique noté H est un pseudo état qui permet de mémoriser le dernier état visité d'un automate pour y retourner ultérieurement.

Indicateur d'historique

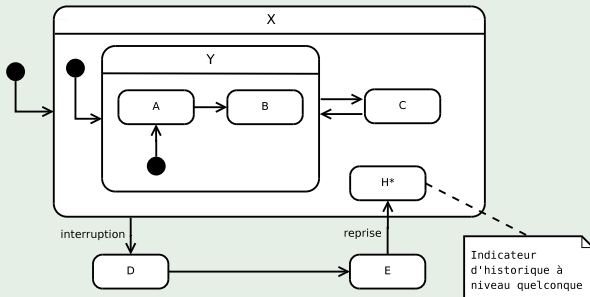


Si une interruption survient, suite à la reprise, l'état du système retournera à celui avant l'interruption.

Indicateurs d'historique

L'indicateur d'historique à niveau quelconque noté H^* est un pseudo état qui permet de mémoriser le dernier état visité, à un état d'imbrication quelconque, pour y retourner ultérieurement.

Indicateur d'historique à niveau quelconque



Si une interruption survient, suite à la reprise, l'état du système retournera à celui avant l'interruption, c'est-à-dire dans l'état A, B ou C.

Automates en parallèle

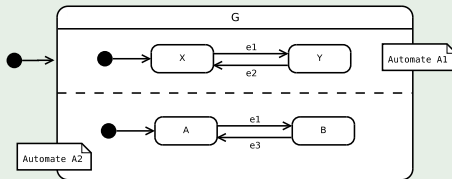
A l'intérieur d'un état, plusieurs automates peuvent s'exécuter en parallèle. Chaque sous-automate a un état initial et un certain nombre d'états finaux.

L'activité d'un tel état se termine lorsque tous les sous-automates arrivent à un état final.

Lorsqu'un événement se produit, toutes les transitions qui peuvent être exécutées sont effectuées.

Automates en parallèle (suite)

Exécution d'automates en parallèles



Il est possible d'aplatir l'automate pour obtenir un comportement équivalent.

Typiquement, les automates en parallèle s'utilisent lorsqu'un objet est formé de la composition ou l'agrégation d'autres objets, en particulier si le comportement de l'objet composite correspond à la mise en parallèle des comportements des composants.

Les aspects dynamiques

Les diagrammes de séquence

Les diagrammes de collaboration

Les diagrammes d'états-transitions

Les diagrammes d'activité

Les diagrammes d'activité

Les diagrammes d'activité permettent de modéliser le comportement d'une méthode ou le déroulement d'un cas d'utilisation, en reprenant un enchaînement d'activités.

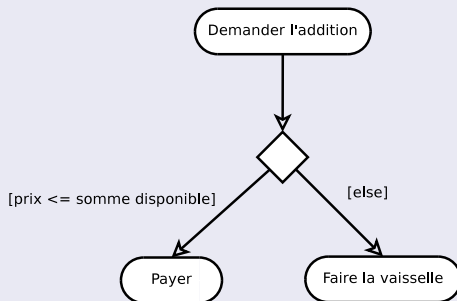
Activité et transition



Transitions conditionnelles

Il est possible de donner des conditions pour le franchissement de transitions.

Transitions conditionnelles

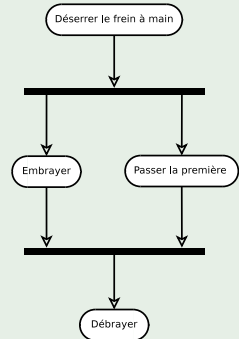


Synchronisation

Il est possible de synchroniser des transitions à l'aide d'une **barre de synchronisation**. Celle-ci permet d'ouvrir et de fermer des branches parallèles au sein d'un flôt d'exécution :

- Les transitions qui partent d'une barre de synchronisation ont lieu en même temps.
- On ne franchit une barre de synchronisation qu'après réalisation de toutes les transitions qui s'y rattachent.

Barre de synchronisation



Couloirs d'activités

```

graph TD
    subgraph Client
        Start(( )) --> DemanderAddition(Demander l'addition)
        RecevoirColis(Recevoir colis) --> ReglerFacture(Regler facture)
    end
    subgraph Fournisseur
        DemanderAddition --> Decision{ }
        Decision -- "[pas en stock]" --> ProduitEpuise(Produit épuisé)
        Decision -- "[en stock]" --> Parallel1[ ]
        Parallel1 --> PreparerCommande(Preparer commande)
        Parallel1 --> EditerFacture(Editer facture)
        PreparerCommande --> Parallel2[ ]
        EditerFacture --> Parallel2
        Parallel2 --> EnvoyerColis(Envoyer colis)
        EnvoyerColis --> RecevoirColis
        EnvoyerColis --> EncaisserFacture(Encaisser facture)
        EncaisserFacture --> End(( ))
        ProduitEpuise --> EndEpuise(( ))
    end
    style Start fill:none,stroke:none
    style End fill:none,stroke:none
    style EndEpuise fill:none,stroke:none
    style Parallel1 fill:none,stroke:none
    style Parallel2 fill:none,stroke:none

```

Bilan du cours

- Cours : apprentissage de la notation UML comme support de l'analyse objet
- TD : démarche de modélisation UML :
 - diagrammes de classes, diagrammes d'objet, OCL (4 séances + colle)
 - diagramme de séquences (1 séance)
 - diagrammes d'états-transitions (3 séances)
 - diagrammes d'activités (1 séance)
- TP : travail sur une étude de cas

Bibliographie

- **UML par la pratique**, P. Roques, Edition Eyrolles
- **UML 2 par la pratique**, P. Roques, Edition Eyrolles
- Cours **Analyse Conception et Validation de Logiciels**, C. Oriat (catherine.oriat@imag.fr), ENSIMAG à Grenoble.
- Site web de l'OMG : [http ://www.omg.org](http://www.omg.org)
- Un site complet sur UML en français : [http ://uml.free.fr](http://uml.free.fr)